PYTHON • LECTURE 04

Dictionary & Set

Python Built-in Collections

Lecture based on the linked Python Full Course video, redesigned for PGD classroom teaching.

Instructor

Syed Uzair Hussain • NED Academy

Focus

Dictionaries • Sets • Methods • Practice

Learning Objectives

After this lecture students should be able to use dictionaries and sets confidently.

01

Understand key-value pairs in dictionaries

02

Create, read, update and nest dictionaries

03

Use important dictionary methods

04

Understand sets and unique elements

05

Use set methods and solve practice tasks

Where We Are in Python Collections

Lists, tuples, dictionaries and sets store multiple values — but each has a different purpose.

LIST

Ordered, mutable
Use [ ]
Example: [1, 2, 3]

TUPLE

Ordered, immutable
Use ( )
Example: (1, 2, 3)

DICT

Key-value pairs
Use {key: value}
Example: {"age": 21}

SET

Unique values
Use { } or set()
Example: {1, 2, 3}

What is a Dictionary?

A dictionary stores data in key-value pairs.

KEY

A unique label used to access data.
Example: "name", "age", "city"

VALUE

The actual data stored against a key.
Example: "Ali", 21, "Karachi"

```
Dictionary = { key : value }
```

Creating a Dictionary

Use curly braces { } and write each item as key: value.

```
info = {
    "name": "Ali",
    "age": 21,
    "marks": 87.5
}

print(info)
```

Keys and Values

The key is used to identify a value.

name

"Ali"

age

21

city

"Karachi"

Each key points to exactly one current value.

Dictionary Can Store Mixed Values

Values can be strings, numbers, lists, tuples or another dictionary.

```
student = {
    "name": "Sara",
    "subjects": ["Python", "DBMS", "AI"],
    "marks": (85, 90, 88),
    "is_passed": True
}

print(student)
```

Accessing Dictionary Values

Use the key to access a value.

```
student = {"name": "Ali", "age": 21, "city": "Karachi"}

print(student["name"])
print(student["city"])
```

Adding and Updating Values

Dictionaries are mutable, so values can be changed after creation.

```
student = {"name": "Ali", "age": 21}

student["age"] = 22         # update
student["city"] = "Karachi"  # add new key

print(student)
```

Nested Dictionary

A dictionary can contain another dictionary inside it.

```
student = {
    "name": "Ayesha",
    "score": {
        "python": 88,
        "database": 91,
        "ai": 86
    }
}

print(student["score"]["python"])
```

Dictionary Methods Overview

Methods help us inspect and update dictionary data.

keys()

all keys

values()

all values

items()

key-value pairs

get()

safe access

update()

add/update data

dict.keys()

Returns all keys in the dictionary.

```
student = {"name": "Ali", "age": 21, "city": "Karachi"}

print(student.keys())
print(list(student.keys()))
```

dict.values()

Returns all values in the dictionary.

```
student = {"name": "Ali", "age": 21, "city": "Karachi"}

print(student.values())
print(list(student.values()))
```

dict.items()

Returns key-value pairs as tuples.

```
student = {"name": "Ali", "age": 21, "city": "Karachi"}

print(student.items())

for key, value in student.items():
    print(key, value)
```

dict.get()

Safer way to access a value by key.

```
student = {"name": "Ali", "age": 21}

print(student.get("name"))
print(student.get("city"))      # None
# print(student["city"])        # KeyError
```

dict.update()

Adds new key-value pairs or updates existing keys.

```
student = {"name": "Ali", "age": 21}

student.update({"city": "Karachi", "age": 22})

print(student)
```

Important Dictionary Rules

These points help beginners avoid common mistakes.

01

Keys should be unique. Duplicate keys overwrite old values.

02

Dictionary values can be duplicated.

03

Dictionaries are mutable.

04

Keys are usually strings, numbers or tuples.

05

Lists cannot be used as dictionary keys.

Dictionary Mini Practice

Create a student profile dictionary.

```
profile = {}
profile["name"] = input("Enter name: ")
profile["age"] = int(input("Enter age: "))
profile["city"] = input("Enter city: ")

print(profile)
```

What is a Set?

A set stores unordered, unique values.

UNIQUE

Repeated values are stored only once.
Useful for removing duplicates.

UNORDERED

Sets do not maintain a reliable index order.
Do not use set[0].

Set = collection of unique immutable elements

Creating a Set

Use curly braces with values, not key-value pairs.

```
numbers = {1, 2, 3, 4}

print(numbers)
print(type(numbers))
```

Duplicate Values in Set

Sets automatically keep only unique values.

```
nums = {1, 2, 2, 3, 3, 3, 4}

print(nums)
```

Creating an Empty Set

Important beginner point: { } creates an empty dictionary, not an empty set.

```
empty_dict = {}
empty_set = set()

print(type(empty_dict))
print(type(empty_set))
```

Set Rules

Sets are mutable, but set elements must be immutable.

01

Set itself can be changed using methods.

02

Elements must be immutable values.

03

Numbers, strings and tuples can be stored.

04

Lists and dictionaries cannot be stored inside a set.

05

Sets are unordered, so indexing does not work.

Set Methods Overview

Common methods used in beginner Python programs.

add()

add value

remove()

remove value

clear()

empty set

pop()

remove random item

union()

combine unique

intersection()

common values

set.add()

Adds one value to the set.

```
subjects = {"Python", "AI"}

subjects.add("Database")

print(subjects)
```

set.remove()

Removes a value from the set. If the value is missing, it gives an error.

```
subjects = {"Python", "AI", "Database"}

subjects.remove("AI")

print(subjects)
```

set.clear()

Removes all items from the set.

```
subjects = {"Python", "AI", "Database"}

subjects.clear()

print(subjects)
```

set.pop()

Removes and returns a random item because sets are unordered.

```
subjects = {"Python", "AI", "Database"}

removed = subjects.pop()

print("Removed:", removed)
print(subjects)
```

set.union()

Combines two sets and returns all unique values.

```
a = {1, 2, 3}
b = {3, 4, 5}

print(a.union(b))
```

set.intersection()

Returns only common values between two sets.

```
a = {1, 2, 3}
b = {3, 4, 5}

print(a.intersection(b))
```

Dictionary vs Set

Both use curly braces, but their purpose is different.

DICTIONARY

Purpose: map keys to values
Syntax: {"key": "value"}
Access: dict["key"]
Duplicate keys: overwritten

SET

Purpose: store unique values
Syntax: {1, 2, 3}
Access: no indexing
Duplicate values: removed

Practice 1 — Word Meanings

Store word meanings in a dictionary.

```
dictionary = {
    "table": ["a piece of furniture", "list of facts and figures"],
    "cat": "a small animal"
}

print(dictionary)
```

Practice 2 — Unique Classrooms

Use a set to find how many unique subjects require separate classrooms.

```
subjects = {
    "python", "java", "C++", "python",
    "javascript", "java", "python", "java", "C++", "C"
}

print(subjects)
print("Classrooms needed:", len(subjects))
```

Practice 3 — Marks Dictionary

Take marks of 3 subjects from the user and store them in a dictionary.

```
marks = {}

marks["Physics"] = int(input("Enter Physics marks: "))
marks["Math"] = int(input("Enter Math marks: "))
marks["Computer"] = int(input("Enter Computer marks: "))

print(marks)
```

Practice 4 — Store 9 and 9.0 Separately

In Python, 9 and 9.0 are treated as equal in a set. Use different data types to store separately.

```
values = {9, "9.0"}

print(values)

# Another possible approach:
values2 = {("int", 9), ("float", 9.0)}
print(values2)
```

Common Beginner Mistakes

Use this slide during live coding.

01

Writing {} and expecting an empty set.

02

Trying to access set values using index.

03

Forgetting quotes around string keys.

04

Using duplicate keys and expecting both to remain.

05

Using list as a dictionary key or set item.

Class Lab Flow

Suggested 45–60 minute classroom practice.

01

Demo dictionary creation and key access.

02

Students solve marks dictionary task.

03

Demo set unique values and empty set.

04

Students solve classroom-subject task.

05

End with 9 vs 9.0 discussion.

Lecture Recap

Key ideas to remember before the next lecture.

DICTIONARY

Key-value structure
Mutable
Best for labeled records

SET

Unique values
Unordered
Best for removing duplicates

PRACTICE

Methods + tasks
Live coding
Build confidence